AgroSmart

Análisis de Algoritmos y Estructuras de Datos

Módulo: Calendario Agrícola y Predicción de Cosechas

1. Sistema de Cache con Diccionario (Tabla Hash)

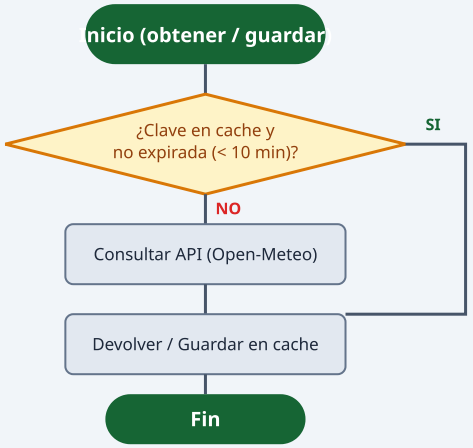
EXPLICACIÓN DEL FUNCIONAMIENTO

El módulo guarda en memoria, dentro de un diccionario (_CACHE), el último resultado obtenido para cada localidad y datos agrometeorológicos. Cuando se solicita el clima o la proyección de cosecha para una zona, primero se revisa si existe un resultado guardado y si no han pasado más de 10 minutos (600 segundos) desde que se registró. Si es así, se devuelve ese valor de inmediato sin consultar la API externa. Si no existe o ya expiró, se consulta el valor real, se guarda en el diccionario junto con la hora actual y se devuelve. El diccionario funciona como una tabla hash, ubicando la clave casi instantáneamente.

PSEUDOCÓDIGO

```
      FUNCION obtener_de_cache(clave) SI clave EXISTE en cache ENTONCES (valor, guardado_en)
= cache[clave] SI (tiempo_actual - guardado_en) < duracion_cache ENTONCES DEVOLVER valor FIN
SI FIN SI DEVOLVER nulo FIN FUNCION FUNCION guardar_en_cache(clave, valor) cache[clave] =
(valor, tiempo_actual) FIN FUNCION
```

DIAGRAMA DE FLUJO



ANÁLISIS DE COMPLEJIDAD

Métrica	Complejidad	Descripción
Complejidad temporal	O(1) en promedio	El diccionario calcula directamente la posición de la clave sin recorrer los demás datos guardados.
Complejidad espacial	O(k)	Donde *k* es el número de localidades distintas consultadas; se guarda una entrada por cada una.

Justificación de su elección: Se eligió un cache basado en diccionario porque las consultas climáticas dependen de un servicio externo que puede ser lento o tener límites de peticiones. Evita peticiones repetidas al probar o ejecutar consultas simultáneas.

2. Recorrido Lineal para el Pronóstico de Varios Días

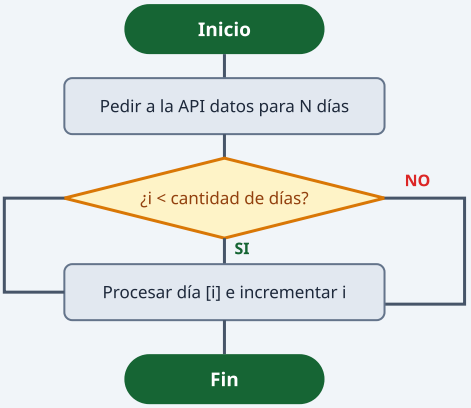
EXPLICACIÓN DEL FUNCIONAMIENTO

Para construir el calendario agrícola y la predicción a varios días, el programa recibe de la API una lista con los datos diarios (temperatura, precipitación, radiación). Recorre esa lista una posición a la vez, de principio a fin. En cada paso crea un objeto Clima/Cosecha con los datos del día y lo agrega a una nueva lista ordenada de pronóstico.

PSEUDOCÓDIGO

```
      FUNCION obtener_pronostico_dias(localidad, dias) datos_diarios =
PEDIR_A_LA_API(localidad, dias) lista_pronostico = [] PARA i DESDE 0 HASTA dias - 1 clima_dia
= CREAR_CLIMA(datos_diarios[i]) AGREGAR clima_dia A lista_pronostico FIN PARA DEVOLVER
lista_pronostico FIN FUNCION
```

DIAGRAMA DE FLUJO



ANÁLISIS DE COMPLEJIDAD

Métrica	Complejidad	Descripción
Complejidad temporal	O(n)	Se revisa cada uno de los *n* días del pronóstico exactamente una vez.
Complejidad espacial	O(n)	Se crea una lista nueva con un objeto por cada día procesado.

Justificación de su elección: Se usó un recorrido lineal simple porque los datos devueltos por la API ya vienen ordenados cronológicamente y la cantidad de días solicitados es pequeña (1 a 16 días).

3. Árbol de Decisión para Generar Alertas Agrícolas

EXPLICACIÓN DEL FUNCIONAMIENTO

La función `generar_alerta` evalúa una serie de condiciones agrometeorológicas en un orden fijo (lluvia fuerte, calor extremo, sequía, heladas). Se detiene en la primera condición que se cumple, devolviendo la alerta correspondiente. Si ninguna condición se cumple, devuelve el estado sin alertas. Este tipo de lógica actúa como un árbol de decisión ordenado por severidad.

PSEUDOCÓDIGO

```
      FUNCION generar_alerta(temperatura, probabilidad_lluvia) lluvia_fuerte =
probabilidad_lluvia >= 70 calor_extremo = temperatura >= 32 sequia = probabilidad_lluvia < 30
Y temperatura >= 30 SI lluvia_fuerte Y calor_extremo ENTONCES DEVOLVER 'Alerta doble: lluvias
fuertes y temperatura muy alta' SINO SI lluvia_fuerte ENTONCES DEVOLVER 'Alerta: se esperan
lluvias fuertes' SINO SI calor_extremo ENTONCES DEVOLVER 'Alerta: temperatura muy alta' SINO
SI sequia ENTONCES DEVOLVER 'Alerta: considera regar tus plantas' SINO DEVOLVER 'Sin alertas
por ahora' FIN SI FIN FUNCION
```

ANÁLISIS DE COMPLEJIDAD

Métrica	Complejidad	Descripción
Complejidad temporal	O(1)	Se evalúa como máximo un número fijo de condiciones (4), sin importar la cantidad de datos.
Complejidad espacial	O(1)	Solo se usan unas pocas variables booleanas temporales, sin estructuras adicionales.

Justificación de su elección: Se usó una cascada de condiciones ordenadas de la más urgente a la menos urgente para que el agricultor reciba un solo mensaje claro y priorizado, evitando alertas duplicadas o confusas.

4. Ordenamiento por Inserción (Insertion Sort)

EXPLICACIÓN DEL FUNCIONAMIENTO

Este algoritmo ordena los días del calendario agrícola o pronóstico según una clave seleccionada (por ejemplo, por temperatura o por probabilidad de lluvia). Construye la lista ordenada un elemento a la vez: toma cada elemento y lo compara con los que ya están ordenados a su izquierda, desplazándolos hacia la derecha hasta ubicarlo en la posición correcta.

PSEUDOCÓDIGO

```
      FUNCION ordenamiento_insercion(lista_climas, clave) PARA i DESDE 1 HASTA
longitud(lista_climas) - 1 actual = lista_climas[i] j = i - 1 MIENTRAS j >= 0 Y
lista_climas[j].clave > actual.clave lista_climas[j + 1] = lista_climas[j] j = j - 1 FIN
MIENTRAS lista_climas[j + 1] = actual FIN PARA DEVOLVER lista_climas FIN FUNCION
```

ANÁLISIS DE COMPLEJIDAD

Métrica	Complejidad	Descripción
Complejidad temporal	$O(n^2)$ peor / O(n) mejor	En el peor caso realiza comparaciones cuadráticas; si la lista está casi ordenada, es lineal.
Complejidad espacial	O(1)	Ordena los elementos dentro de la misma lista sin requerir memoria extra.

Justificación de su elección: Dado que el número de días a ordenar en el calendario agrícola es reducido (generalmente entre 7 y 16 días), el algoritmo de inserción es altamente eficiente, no requiere memoria auxiliar y funciona de forma óptima para arreglos pequeños o parcialmente ordenados.